

SIMULACRO DE PARCIAL

Tema: Construyendo la UdeA del futuro

La Universidad de Antioquia ha iniciado la construcción de las bases estratégicas de su **Plan de Desarrollo Institucional 2028–2038**, un proceso que busca recoger las ideas, necesidades y propuestas de diferentes integrantes de la comunidad universitaria. Como parte de este proceso, se desea desarrollar una pequeña aplicación que permita **simular algunos de los mecanismos utilizados para recibir y analizar la información enviada por los participantes**.

La aplicación no pretende almacenar información en una base de datos. Su propósito es **reconocer si la información suministrada cumple con las condiciones establecidas por la Universidad**.

Condiciones generales

El programa debe desarrollarse en **C++** y debe estar organizado mediante **funciones**.

No se indicará en cada punto qué mecanismo, estructura o técnica debe utilizarse. El estudiante deberá **analizar el problema, identificar qué comportamiento debe reconocer el programa y seleccionar la estrategia de solución que considere apropiada**.

Se espera que el código sea claro, modular y permita probar diferentes casos.

Punto 1. Registro de participantes

Para participar en la construcción del Plan de Desarrollo, la Universidad habilitará un formulario en el que cada persona deberá registrar un código de participación.

El código permitirá identificar de manera sencilla el tipo de participante y el número asignado dentro del proceso.

Se ha definido que los códigos tendrán las siguientes características:

- Deben comenzar con una de las siguientes categorías:
 - EST
 - PRO
 - ADM
 - EGR

- Después de la categoría debe aparecer un guion -.
- Finalmente debe aparecer un número de **cuatro dígitos**.
- No se permiten espacios.
- No se permiten caracteres adicionales.

Ejemplos:

EST-1025

PRO-0007

ADM-4521

EGR-1234

Mientras que los siguientes registros no deben ser aceptados:

EST1025

EST-125

EST-12345

est-1025

PRO_1234

ADM-12A4

Requerimiento

Desarrolle una función que permita determinar si un código de participación cumple con las condiciones establecidas.
El programa debe solicitar varios códigos y mostrar para cada uno:

Código: EST-1025

Resultado: ACEPTADO

o

Código: EST-12A5

Resultado: RECHAZADO

El estudiante debe determinar qué mecanismo resulta apropiado para reconocer este tipo de información.

Punto 2. Registro de propuestas

Durante el proceso de construcción del Plan de Desarrollo, una persona puede registrar una propuesta relacionada con

diferentes aspectos de la Universidad.

Para facilitar el análisis, cada propuesta tendrá una estructura sencilla:

CATEGORIA:PROPUESTA

Las categorías permitidas son:

ACADEMICA

INVESTIGACION

BIENESTAR

INFRAESTRUCTURA

Por ejemplo:

ACADEMICA:Fortalecer los cursos de programación

Sin embargo, el sistema debe comprobar que la estructura general de la información sea correcta.

Una propuesta válida debe:

1. Tener una categoría válida.
2. Contener :.
3. Tener un texto después de :.
4. No terminar inmediatamente después de :.
5. No contener otra categoría antes de la primera :.

Requerimiento

Construya una solución que permita analizar una propuesta y determinar si puede ser procesada por el sistema.

El programa debe permitir probar diferentes propuestas y mostrar si cada una puede continuar al siguiente proceso.

Por ejemplo:

ACADEMICA:Fortalecer los cursos de programación

→ PROPUESTA ACEPTADA

BIENESTAR:

→ PROPUESTA RECHAZADA

DEPORTE:Crear nuevos espacios

→ PROPUESTA RECHAZADA

Importante

La Universidad podría incorporar nuevas categorías posteriormente. Por esta razón, procure que la solución esté organizada de manera que las reglas puedan modificarse sin tener que reconstruir todo el programa.

Punto 3. Estructura de las propuestas

Durante el análisis de las propuestas se permitirá que una propuesta contenga otras propuestas relacionadas. Por ejemplo, una propuesta general podría organizarse así:

```
{  
  propuesta  
  {  
    propuesta  
    propuesta  
  }  
}
```

También podría existir una estructura más profunda:

```
{  
  propuesta  
  {  
    propuesta  
    {  
      propuesta  
    }  
  }  
}
```

El sistema debe determinar si los símbolos utilizados para delimitar estas estructuras están correctamente organizados.

Se consideran correctas, por ejemplo:

```
{propuesta}  
{{propuesta}}  
{propuesta{propuesta}}  
{{propuesta}{propuesta}}
```

Pero deben rechazarse:

```
{propuesta  
propuesta}  
{propuesta}  
{{propuesta}  
}{propuesta{
```

Requerimiento

Desarrolle una función que permita determinar si una estructura recibida por el sistema está correctamente delimitada.

La solución debe soportar **diferentes niveles de anidamiento**, pues no se conoce de antemano cuántos niveles puede tener una propuesta.

Nota: No se especifica la estructura de datos que debe utilizar para resolver este punto. Analice el comportamiento requerido y determine cuál resulta apropiada.

Punto 4. Construcción de una propuesta completa

Finalmente, la Universidad desea realizar una prueba del proceso completo.

Una participación estará compuesta por:
Código + Propuesta + Estructura

Por ejemplo:

EST-1025

ACADEMICA:Fortalecer los cursos de programación

```
{propuesta{propuesta}}
```

El programa deberá analizar la información recibida y determinar si puede ser aceptada.

Para que una participación sea aceptada:

- El código debe cumplir las condiciones establecidas.
- La categoría de la propuesta debe ser válida.
- La propuesta debe contener información.
- La estructura de delimitadores debe estar correctamente organizada.

Requerimiento

Construya un programa que permita ingresar la información de un participante y genere un resultado como:

```
=====
RESULTADO DE PARTICIPACIÓN
=====
```

Código: EST-1025

Código ACEPTADO

Propuesta: ACADEMICA:Fortalecer los cursos

Propuesta ACEPTADA

Estructura: {propuesta{propuesta}}

Estructura ACEPTADA

PARTICIPACIÓN ACEPTADA

En caso contrario:

PARTICIPACIÓN RECHAZADA

Condición adicional

Cada una de las validaciones debe estar implementada mediante **funciones independientes**.

El main() debe encargarse principalmente de:

- solicitar la información;
- llamar las funciones;
- presentar los resultados.

Evite concentrar toda la lógica del problema dentro de main().